

✅ Step-by-Step Jenkins Job Setup

🔧 Prerequisites

- Jenkins installed and running
- Java and Maven installed on Jenkins server/agent
- Git installed
- Jenkins plugins:
 - **Git Plugin**
 - **Maven Integration Plugin**
 - (Optional) **SonarQube Scanner Plugin**
 - (Optional) **Artifactory Plugin**

🧩 Step 1: Create a New Jenkins Job

1. Go to Jenkins dashboard → **New Item**
2. Enter name: `mahalogin-pipeline`
3. Select **Freestyle project** → Click **OK**

🔗 Step 2: Configure Source Code Management

1. Go to **Source Code Management** → Select **Git**
2. **Repository URL:** `https://github.com/AbdulRehaman082493/mahalogin.git`
3. **Branches to build:**
`*/main`

🕒 Step 3: Configure Build Triggers

Choose one of the following:

- **Poll SCM:**

H/15 * * * * # Every 15 mins

- **GitHub webhook** (preferred for faster trigger):
- In GitHub, go to repo → Settings → Webhooks → Add:

`http://<jenkins-url>/github-webhook/`

Step 4: Configure Build Environment (Optional)

- ☒ Check **Delete workspace before build starts** (clean build)
- ☒ Add environment variables if needed

Step 5: Add Build Step – Maven Build

1. **Build** → **Add build step** → **Invoke top-level Maven targets**
2. Set:
 - **Goals:**
clean install
 - **POM:** pom.xml

(Optional) **Step 6: Code Analysis using SonarQube**

If SonarQube is configured:

1. Add **Build Step** → **Invoke SonarQube Scanner**
2. Set project properties or use `sonar-project.properties`

Step 7: Archive the Build Artifacts

1. **Post-build Actions** → **Archive the artifacts**
2. Set: `target/*.war`

(Optional) **Step 8: Upload to JFrog Artifactory**

If integrated:

1. Add **Post-build Action** → **Deploy artifacts to Artifactory**
2. Configure:
 - Artifactory server
 - Target repository
 - Artifact pattern: `target/*.war`

Step 9: Save and Run

- Click **Save**
- Click **Build Now**
- Observe Console Output to verify the flow

Summary Flow

Step	Action
1	Jenkins polls GitHub repo or listens to webhook
2	Clones <code>main</code> branch of <code>mahalogin.git</code>
3	Cleans and compiles Java source code
4	Runs unit tests

- 5 Generates **.war** artifact
- 6 (Optional) Runs SonarQube code analysis
- 7 Archives **.war** file
- 8 (Optional) Uploads to JFrog repo

Dashboard > Manage Jenkins > System >

JFrog

☐ Use the Credentials Plugin ?

JFrog Platform Instances

List of JFrog platform instances that projects will want to deploy artifacts and build info

≡ JFrog instance details

Instance ID ?

myjfroginstance01

JFrog Platform URL ?

http://3.138.105.75:8082

Advanced Configuration

▼

Default Deployer Credentials

Username ?

admin

Password ?

.....

Found JFrog Artifactory 7.9.2 at http://3.138.105.75:8082/artifactory

JFrog Distribution not found at http://3.138.105.75:8082/distribution

Test Connection

☐ Use Different Resolver Credentials

Add JFrog Platform Instance

Maven

Name

mymaven01

☒ Install automatically ?

Install from Apache

Version

3.9.9

Add Installer

Add Maven

Save

Apply

.Jenkins :

- General
- Source Code Management
- Triggers
- Environment
- Build Steps
- Post-build Actions

Source Code Management

Connect and manage your code repository to automatically pull the latest code for your builds.

☐ None

☒ Git ?

Repositories ?

Repository URL ?

https://github.com/AbdulRehaman082493/mahalogin.git

Credentials ?

- none -

+ Add

Advanced

Hide the menu for JenkinsNode01

Add Repository

Branches to build ?

Branch Specifier (blank for 'any') ?

*/master

Add Branch

Repository browser ?

(Auto)

Additional Behaviours

Add

Configure

- General
- Source Code Management
- Triggers
- Environment
- Build Steps
- Post-build Actions

Triggers

Set up automated actions that start your build based on specific events, like code changes or scheduled times.

- Trigger builds remotely (e.g., from scripts)
- Build after other projects are built
- Build periodically
- Enable Artifactory trigger
- GitHub hook trigger for GITScm polling
- ☒ Poll SCM

Schedule

H/15 * * * *

Would last have run at Tuesday, May 13, 2025 at 4:04:00 AM Coordinated Universal Time; would next run at Tuesday, May 13, 2025 at 4:19:00 AM Coordinated Universal Time.

- Ignore post-commit hooks

Environment

Configure settings and variables that define the context in which your build runs, like credentials, paths, and global parameters.

- Delete workspace before build starts
- Use secret text(s) or file(s)
- Provide Configuration files
- ☒ Add timestamps to the Console Output
- Ant/Ivy-Artifactory Integration
- Generic-Artifactory Integration
- Gradle-Artifactory Integration
- Inspect build log for published build scans
- Maven3-Artifactory Integration
- Terminate a build if it's stuck
- With Ant

Configure

- General
- Source Code Management
- Triggers
- Environment
- Build Steps
- Post-build Actions

- Inspect build log for published build scans
- Maven3-Artifactory Integration
- Terminate a build if it's stuck
- With Ant

Build Steps

Automate your build process with ordered tasks like code compilation, testing, and deployment.

Invoke top-level Maven targets

Maven Version
(Default)

Goals
clean install

Advanced

Add build step

Post-build Actions

Define what happens after a build completes, like sending notifications, archiving artifacts, or triggering other jobs.

Archive the artifacts

Files to archive
target/*.war

Advanced

Add post-build action

Save Apply

Timestamps

view as plain text

System clock time

Use browser timezone

Elapsed time

None

Progress (1): 109/230 kB
Progress (1): 114/230 kB
Progress (1): 118/230 kB
Progress (1): 122/230 kB
Progress (1): 126/230 kB
Progress (1): 130/230 kB
Progress (1): 134/230 kB
Progress (1): 138/230 kB
Progress (1): 142/230 kB
Progress (1): 146/230 kB
Progress (1): 150/230 kB
Progress (1): 154/230 kB
Progress (1): 159/230 kB
Progress (1): 163/230 kB
Progress (1): 167/230 kB
Progress (1): 171/230 kB
Progress (1): 175/230 kB
Progress (1): 179/230 kB
Progress (1): 183/230 kB
Progress (1): 187/230 kB
Progress (1): 191/230 kB
Progress (1): 195/230 kB
Progress (1): 200/230 kB
Progress (1): 204/230 kB
Progress (1): 208/230 kB
Progress (1): 212/230 kB
Progress (1): 216/230 kB
Progress (1): 220/230 kB
Progress (1): 224/230 kB
Progress (1): 228/230 kB
Progress (1): 230 kB

Downloaded from central: <https://repo.maven.apache.org/maven2/org/codehaus/plexus/plexus-utils/3.0.5/plexus-utils-3.0.5.jar> (230 kB at 8.5 MB/s)
[1;34mINFO [m] Installing /var/lib/jenkins/workspace/mahalogin-pipeline/target/mahalogin-3.0.war to
/var/lib/jenkins/.m2/repository/com/maha/mahalogin/3.0/mahalogin-3.0.war
[1;34mINFO [m] Installing /var/lib/jenkins/workspace/mahalogin-pipeline/pom.xml to /var/lib/jenkins/.m2/repository/com/maha/mahalogin/3.0/mahalogin-3.0.pom
[1;34mINFO [m] [1m-----[m]
[1;34mINFO [m] [1;32mBUILD SUCCESS [m
[1;34mINFO [m] [1m-----[m]
[1;34mINFO [m] Total time: 11.794 s
[1;34mINFO [m] Finished at: 2025-05-13T04:15:19Z
[1;34mINFO [m] [1m-----[m]
Archiving artifacts
Finished: SUCCESS

Dashboard > mahalogin-pipeline > #1

Status

</> Changes

Console Output

Edit Build Information

Delete build '#1'

Timings

Git Build Data

✓ #1 (May 13, 2025, 4:15:02 AM)

Build Artifacts

mahaLogin-3.0.war 16.64 MiB [view](#)

Started by user Abdul Rehaman Shaik

This run spent:

- 55 ms waiting;
- 16 sec build duration;
- 16 sec total from scheduled to completion.

git

Revision: 9044a5d168d74bef4371f71315d743b03ad3ba0

Repository: <https://github.com/AbdulRehaman082493/mahalogin.git>

- refs/remotes/origin/master

</>

No changes.

Add description

Keep this build fo

Started 2 min 51 :
Took 16 sec on bi

b. Night Builds (Clean Build on SIT/UAT)

- **Time:** Scheduled at 2:00 AM
- **Triggered by:** Cron `2 * * * *` (or similar)
- **Steps:**
 1. Clean Workspace
 2. Clone from SCM
 3. Maven Clean + Compile + Install
 4. Generate Artifact
 5. Push artifact to remote repo (e.g., JFrog)

6. (Optional) Run Sonar analysis
7. Run functional test cases
8. (Optional) Run performance test cases (if needed)

Jenkins Night Build (Clean Build) Setup — SIT/UAT

Objective:

Schedule a nightly job in Jenkins that performs a clean build from Git, compiles and tests the Java project using Maven, and pushes the artifact to JFrog Artifactory.

Step 1: Create a Freestyle Jenkins Job

1. Go to **Jenkins Dashboard** → Click **New Item**
2. Enter name: `night-build-mahalogin-SIT`
3. Choose **Freestyle project**
4. Click **OK**

New Item

Enter an item name

night-build-mahalogin-SIT

» A job already exists with the name 'night-build-mahalogin-SIT'

Select an item type



Freestyle project

Classic, general-purpose job type that checks out from up to one SCM, executes build steps serially, followed by post-build steps like archiving artifacts and sending email notifications.



Maven project

Build a maven project. Jenkins takes advantage of your POM files and drastically reduces the configuration.



Pipeline

Orchestrates long-running activities that can span multiple build agents. Suitable for building pipelines (formerly known as workflows) and/or organizing complex activities that do not easily fit in free-style job type.



Multi-configuration project

Suitable for projects that need a large number of different configurations, such as testing on multiple environments, platform-specific builds, etc.



Folder

Creates a container that stores nested items in it. Useful for grouping things together. Unlike view, which is just a filter, a folder creates a separate namespace, so you can have multiple things of the same name as long as they are in different folders.



Multibranch Pipeline

Creates a set of Pipeline projects according to detected branches in one SCM repository.



Organization Folder

Creates a set of multibranch project subfolders by scanning for repositories.

If you want to create a new item from other existing, you can use this option:

Copy from

Type to autocomplete

OK

Step 2: Source Code Management

1. Select **Git** under **Source Code Management**
2. **Repository URL:** `https://github.com/AbdulRehaman082493/mahalogin.git`
3. **Credentials:** Add GitHub credentials if private repo
4. **Branch Specifier:** `*/SIT`

Dashboard > night-build-mahalogin-SIT > Configuration

Configure

- General
- Source Code Management
- Triggers
- Environment
- Build Steps
- Post-build Actions

General

Enabled 

Description

Plain text [Preview](#)

☐ Discard old builds ?

☐ GitHub project

☐ This project is parameterized ?

☐ Throttle builds ?

☐ Execute concurrent builds if necessary ?

☒ Restrict where this project can be run ?

Label Expression ?

mylabel01

Label mylabel01 matches 1 node. Permissions or other restrictions provided by plugins may further reduce that list.

Advanced 

Source Code Management

Connect and manage your code repository to automatically pull the latest code for your builds.

☐ None

☒ Git ?

Repositories ?

Repository URL ?

https://github.com/AbdulRehman082493/mahalogin.git

Credentials ?

- none -

+ Add

Step 3: Trigger by Cron (Schedule at 2:00 AM)

1. Go to **Build Triggers**
2. Select **Build periodically**
3. Add the following CRON: 0 2 * * *

 Triggers the build every day at **2:00 AM**

Dashboard > night-build-mahalogin-SIT > Configuration

Configure

- General
- Source Code Management
- Triggers**
- Environment
- Build Steps
- Post-build Actions

Triggers

Set up automated actions that start your build based on specific events, like code changes or scheduled times.

- ☐ Trigger builds remotely (e.g., from scripts) ?
- ☐ Build after other projects are built ?
- ☒ **Build periodically** ?

Schedule ?

H 2 * * *

Would last have run at Tuesday, May 13, 2025 at 2:12:00 AM Coordinated Universal Time; would next run at Wednesday, May 14, 2025 at 2:12:00 AM Coordinated Universal Time.

- ☐ Enable Artifactory trigger
- ☐ GitHub hook trigger for GITScm polling ?
- ☐ Poll SCM ?

Environment

Configure settings and variables that define the context in which your build runs, like credentials, paths, and global parameters.

- ☒ **Delete workspace before build starts**

Advanced ▾

- ☐ Use secret text(s) or file(s) ?
- ☐ Provide Configuration files ?
- ☐ Add timestamps to the Console Output
- ☐ Ant/Ivy-Artifactory Integration
- ☐ Generic-Artifactory Integration
- ☐ Gradle-Artifactory Integration
- ☐ Inspect build log for published build scans
- ☐ Maven3-Artifactory Integration
- ☐ Terminate a build if it's stuck
- ☐ With Ant ?

Build Steps

Automate your build process with ordered tasks like code compilation, testing, and deployment.

Step 4: Clean Workspace Before Build

1. Go to Build Environment
2. Check ☒ Delete workspace before build starts

Step 5: Build Configuration with Maven

1. Go to Build → Add build step → Invoke top-level Maven targets
2. Fill in:
 - Goals: clean install
 - POM: pom.xml

Step 6: Archive Artifact

1. Go to Post-build Actions
2. Select Archive the artifacts
3. Set: `target/*.war`

Step 7: Push Artifact to JFrog Artifactory (Optional)

1. Go to Post-build Actions → Deploy artifacts to Artifactory
2. Set:
 - Artifact Path: `target/*.war`
 - Target Repo: Example: `libs-release-local`
3. Configure your JFrog credentials and Artifactory server in Jenkins global config first.

Step 8: (Optional) SonarQube Code Analysis

If SonarQube is integrated:

1. Add Build Step → Invoke SonarQube Scanner
2. Provide:
 - Project Key
 - Source Path
 - OR use `sonar-project.properties` in root directory

Step 9: Run Functional Tests

1. Add Build Step → Execute Shell (or Maven goals)
2. Example command:

```
echo "Running Functional Test Cases..."
```

mvn verify

Step 10: (Optional) Run Performance Test Cases

If using JMeter or other tool:

1. Add **Build Step** → **Execute Shell**
2. Example: `jmeter -n -t testplan.jmx -l results.jtl`

Summary Table

Step	Task	Description
1	Cron trigger	<code>0 2 * * *</code> to run at 2:00 AM
2	Git checkout	From <code>SIT</code> or <code>UAT</code> branch
3	Clean workspace	Ensure clean environment
4	Maven build	Run <code>clean install</code>
5	Archive <code>.war</code>	Save artifacts

- | | | |
|---|-----------------------|-------------------------|
| 6 | Push to Artifactory | Optional for deployment |
| 7 | SonarQube
Analysis | Optional code quality |
| 8 | Functional Tests | Ensure working build |
| 9 | Performance Tests | Optional if configured |

Steps: JFrog Artifactory Integration with Jenkins for Artifact Upload

Goal:

After building the `mahalogin` project with Maven, automatically **upload the `.war` file to JFrog Artifactory** using Jenkins.

Prerequisites:

- Jenkins installed and running
- JFrog Artifactory server running (OSS or Pro)
- Artifactory URL & credentials (username/API key/password)
- Jenkins plugins installed:
 - **Artifactory Plugin**
 - **Maven Integration Plugin**
- `.m2/settings.xml` (optional) for authenticated deploys

Step-by-Step Configuration:

◆ Step 1: Configure Artifactory in Jenkins

1. Go to **Jenkins Dashboard** → **Manage Jenkins**
2. Click **Configure System**
3. Scroll to **JFrog Artifactory**
4. Click **Add Artifactory Server**
 - **Server ID:** `jfrog-sit`
 - **URL:** `http://<your-artifactory-host>:8081/artifactory`

5. Add **Credentials**:

- Username/API Key + Password

6. Click **Test Connection** → ☒ Should be successful

◆ **Step 2: Create a Jenkins Job for SIT/UAT Build**

1. Go to **New Item** → Enter name: `SIT-night-build-artifact`

2. Choose **Freestyle project**

3. Under **Source Code Management**, configure Git:

Repo URL: `https://github.com/AbdulRehaman082493/mahalogin.git`

Branch: `*/SIT` or `*/UAT`

◆ **Step 3: Add Maven Build Step**

1. Under **Build Environment**, check ☒ **Delete workspace before build**

2. Under **Build** → **Add build step** → **Invoke top-level Maven targets**

3. Set:

- **Goals**: clean install deploy
- **POM**: pom.xml

`deploy` phase will push the `.war` to Artifactory (requires distributionManagement section in POM or global Jenkins config override)

◆ **Step 4: Use Jenkins Artifactory Plugin (Optional Method)**

Instead of direct Maven deploy, you can use:

1. Under **Post-build Actions** → Select:

Deploy artifacts to Artifactory

2. Configure:

- **Repository Key:** `mySITrepo`
- **Artifact Pattern:** `target/*.war`
- **Target Path:** `libs-release-local/com/mahalogin/`

◆ Step 5: SCM Polling or Scheduling

Under **Build Triggers**:

- Use Cron for nightly builds: `0 2 * * *` # Every day at 2:00 AM

◆ Step 6: Save and Run Job

1. Click **Save**
2. Click **Build Now**
3. Go to **Console Output** and ensure:
 - Build success
 - `.war` file uploaded to Artifactory under `mySITrepo`

Artifactory Repo Pattern Reference

Setting	Example
Target Repo	<code>mySITrepo</code>

Pattern `target/*.war`

Upload Location `/libs-release-local/`

Deploy Method `mvn deploy` or Plugin

Dashboard > night-build-mahalogin-SIT > Configuration

Configure

- General
- Source Code Management
- Triggers
- Environment**
- Build Steps
- Post-build Actions

Artifactory Configuration

Artifactory server

myfroginstance01 http://3.138.105.75:8082/artifactory

Target releases repository ?

mytestrepo [Select from List](#)

Target snapshots repository ?

mytestrepo [Select from List](#)

[Refresh Repositories](#)

☐ Override default credentials

☐ Resolve artifacts from Artifactory ?

☐ Override build name ?

☒ **Deploy artifacts to Artifactory** ?

Include Patterns ?

Exclude Patterns ?

/.war=>\${BUILD_NUMBER}

☒ **Filter excluded artifacts from build info** ?

Deployment properties ?

☒ **Capture and publish build info** ?

☐ Include environment variables ?

☐ Record implicit project dependencies and build-time dependencies ?

☐ Discard old builds from Artifactory (requires Artifactory Pro) ?

Project ?

Jenkins

Dashboard > night-build-mahalogin-SIT > #1

Status **#1 (May 13, 2025, 4:40:28 AM)** Add description Keep this build fo

</> Changes

Console Output

Edit Build Information

Delete build '#1'

Timings

Git Build Data

Artifactory Build Info

Artifactory Release Promotion

Started by user [Abdul Rehaman Shaik](#)

Started 52 sec ago
Took 22 sec on NodeC

This run spent:

- 4 ms waiting;
- 22 sec build duration;
- 22 sec total from scheduled to completion.

Revision: 9044a5d168d74bef4371f71315d743b03ad3ba0
Repository: <https://github.com/AbdulRehaman082493/mahalogin.git>

- refs/remotes/origin/master

Artifactory Build Info

</> No changes.

✅ 3. Jenkins Backup Strategy Using ThinBackup Plugin

🔧 Purpose:

To **regularly back up Jenkins** configurations, job details, build results, user content, and plugins — and store them in a defined backup folder.

🧩 Step-by-Step Backup Configuration

♦ Step 1: Prepare Backup Directory

SSH into your Jenkins server and create the backup folder:

```
sudo mkdir -p /home/ubuntu/myjenkinsbackup
sudo chown jenkins:jenkins /home/ubuntu/myjenkinsbackup
```

```
(base) abdulrehamanshaik@MacBookPro Linux % ssh jenkinsmaster@74.235.253.152
jenkinsmaster@74.235.253.152's password:
Welcome to Ubuntu 22.04.5 LTS (GNU/Linux 6.8.0-1028-azure x86_64)
```

- * Documentation: <https://help.ubuntu.com>
- * Management: <https://landscape.canonical.com>
- * Support: <https://ubuntu.com/pro>

System information as of Tue May 13 05:07:50 UTC 2025

```
System load: 0.0          Processes:          128
Usage of /: 13.1% of 28.89GB  Users logged in:    0
Memory usage: 13%          IPv4 address for eth0: 10.0.0.4
Swap usage: 0%
```

* Strictly confined Kubernetes makes edge and IoT secure. Learn how MicroK8s just raised the bar for easy, resilient and secure K8s cluster deployment.

<https://ubuntu.com/engage/secure-kubernetes-at-the-edge>

Expanded Security Maintenance for Applications is not enabled.

4 updates can be applied immediately.
To see these additional updates run: `apt list --upgradable`

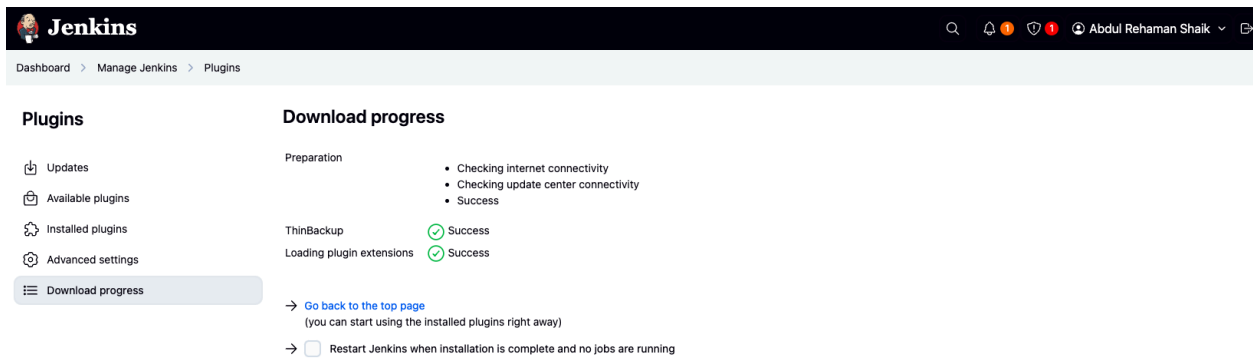
2 additional security updates can be applied with ESM Apps.
Learn more about enabling ESM Apps service at <https://ubuntu.com/esm>

New release '24.04.2 LTS' available.
Run 'do-release-upgrade' to upgrade to it.

```
Last login: Mon May 12 19:22:25 2025 from 98.156.1.186
jenkinsmaster@JenkinsMaster:~$ sudo -i
root@JenkinsMaster:~# sudo mkdir -p /home/ubuntu/myjenkinsbackup
sudo chown jenkins:jenkins /home/ubuntu/myjenkinsbackup
root@JenkinsMaster:~#
```

♦ Step 2: Install ThinBackup Plugin

1. Go to **Jenkins Dashboard** → **Manage Jenkins**
2. Click **Manage Plugins**
3. In the **Available** tab, search for **ThinBackup**
4. Check it and click **Install without restart**



♦ Step 3: Configure ThinBackup

1. Go to **Manage Jenkins** → **ThinBackup**

2. Click ThinBackup Settings

Not Secure 74.235.253.152:8080/manage/configure

Dashboard > Manage Jenkins > System > Backup build results 1/1

ThinBackup Configuration

Backup directory ?
/home/ubuntu/myjenkinsbackup

Backup schedule for full backups ?
H 3 * * 1

Backup schedule for differential backups ?
H 5 * * 2-7

Max number of backup sets ?
10

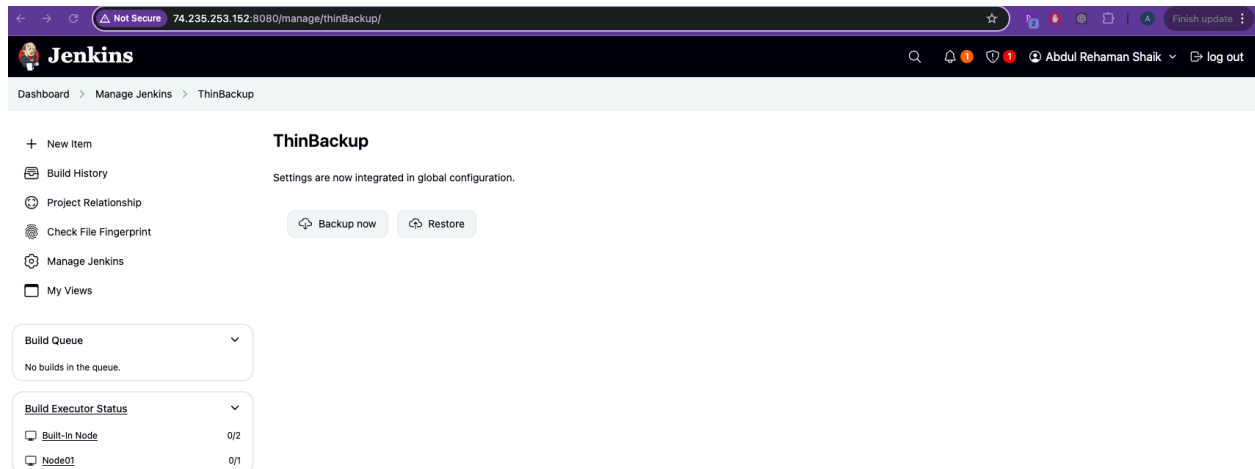
Files excluded from backup (regular expression) ?

☒ Wait until Jenkins is idle to perform a backup ?
Force Jenkins to quiet mode after specified minutes ?
120

☒ Backup build results ?
☒ Backup build archive ?
☒ Backup only builds marked to keep ?

☒ Backup userContent folder ?
☒ Backup next build number file ?
☒ Backup plugins archives ?
☒ Backup config-history folder ?
☐ Backup additional files ?
☐ Clean up differential backups ?
☐ Move old backups to ZIP files ?
☒ Stop the backup as soon as an exception occurs in the file handling ?

Build-timeout Plugin > BuildStep Action



3. Configure the following:

Field	Value
Backup directory	/home/ubuntu/myjenkinsback up
Full backup schedule	H 3 * * 1 (Every Monday at 3 AM)
Differential backup	H 5 * * 2-7 (Tue–Sun at 5 AM)
Maximum number of backups	e.g., 10
Backup build results	☒

Backup archived artifacts	☒
Backup user content	☒
Backup build nodes config	☒
Backup job configs	☒
Backup plugins	☒
Backup next build number	☒

☒ Also check: **Cleanup old backup directories (keep zip)**

What Gets Backed Up

- ☒ Build results (build logs, output)
- ☒ Archived artifacts (***.war**, ***.jar**, etc.)
- ☒ User content folder (**/userContent**)
- ☒ Plugins
- ☒ Job configurations (**config.xml**)
- ☒ Global Jenkins config (**config.xml**, **credentials.xml**)
- ☒ Next build number tracking

Scheduling Reference (CRON Format):

Schedule Type	CRON	Meaning
Full Backup	H 3 * * 1	Every Monday at 3:00 AM
Diff Backup	H 5 * * 2-7	Tue to Sun at 5:00 AM

Use **H** instead of fixed minutes for load balancing across Jenkins masters.

4. Jenkins Restore Strategy with ThinBackup

When needed, you can **restore Jenkins** from a backup ZIP created by ThinBackup.

Step-by-Step Restore Process

◆ Step 1: Go to Restore Menu

1. Jenkins Dashboard → **Manage Jenkins**
2. Click **ThinBackup**
3. Choose **Restore**

◆ Step 2: Choose a Backup Version

1. You'll see a list of **available backup timestamps**
2. Select a backup version (latest or specific date)
3. Click **Restore**

◆ Step 3: Restart Jenkins

- After restore, **restart Jenkins** manually for all restored configs and plugins to take effect:

```
sudo systemctl restart jenkins
```

What Gets Restored

Component	Restored
Jenkins plugins	✓
Job configurations	✓
Build numbers	✓
Global settings	✓
Archived artifacts	✓
Build logs	✓



Notes and Best Practices

- ✓ Store backup folder on **external storage** or mount to **S3/NFS** if needed.
- ✓ Backup directory should have **write access** by the **jenkins** user.
- ✗ Avoid restoring during an active build — stop Jenkins if needed before restore.
- 📦 ThinBackup creates backup as **.zip** — good for offline and portable recovery.

Last login: Mon May 12 19:22:25 2025 from 98.156.1.186

```
jenkinsmaster@JenkinsMaster:~$ sudo -i
```

```
root@JenkinsMaster:~# sudo mkdir -p /home/ubuntu/myjenkinsbackup
```

```
sudo chown jenkins:jenkins /home/ubuntu/myjenkinsbackup
```

```
root@JenkinsMaster:~# ls -al /home/ubuntu/myjenkinsbackup
```

```
total 12
```

```
drwxr-xr-x 3 jenkins jenkins 4096 May 13 05:23 .
```

```
drwxr-xr-x 3 root   root   4096 May 13 05:08 ..
```

```
drwxr-xr-x 7 jenkins jenkins 4096 May 13 05:23 FULL-2025-05-13_05-23
```

```
root@JenkinsMaster:~# ls -al /home/ubuntu/myjenkinsbackup
```

```
total 16
```

```
drwxr-xr-x 4 jenkins jenkins 4096 May 13 05:24 .
```

```
drwxr-xr-x 3 root   root   4096 May 13 05:08 ..
```

drwxr-xr-x 7 jenkins jenkins 4096 May 13 05:23 FULL-2025-05-13_05-23
drwxr-xr-x 7 jenkins jenkins 4096 May 13 05:24 FULL-2025-05-13_05-24
root@JenkinsMaster:~#

Not Secure74.235.253.152:8080

Finish update

Jenkins

Abdul Rehaman Shaiklog out

Dashboard

+ New Item

Build History

Project Relationship

Check File Fingerprint

Manage Jenkins

My Views

Build Queue

No builds in the queue.

Build Executor Status

Built-in Node0/2

Node010/1

All+

S	W	Name ↓	Last Success	Last Failure	Last Duration	
✓	☀	JobA	9 hr 41 min #1	N/A	19 sec	▶
✓	☀	mahalogin-pipeline-Day-Builds	1 hr 9 min #1	N/A	16 sec	▶
✓	☀	night-build-mahalogin-SIT	37 min #3	N/A	13 sec	▶

Icon: S M L

Add description

Deleted JobA

- + New Item
- 📄 Build History
- 🕒 Project Relationship
- 🔍 Check File Fingerprint
- ⚙️ Manage Jenkins
- 📁 My Views

Build Queue ▾

No builds in the queue.

Build Executor Status ▾

📄 Built-in Node	0/2
📄 Node01	0/1

Restore Configuration

Restore options

Restore backup from: ?

2025-05-13 05:24 ▾

- ☐ Restore next build number file (if found in backup) ?
- ☐ Restore plugins ?

Restore

- + New Item
- Build History
- Project Relationship
- Check File Fingerprint
- Manage Jenkins
- My Views
- Build Queue

No builds in the queue.
- Build Executor Status

Built-in Node

0/2

Node01

0/1

All

+

S	W	Name ↓	Last Success	Last Failure	Last Duration	
		JobA	N/A	N/A	N/A	
		mahalogin-pipeline-Day-Builds	1 hr 13 min #1	N/A	16 sec	
		night-build-mahalogin-SiT	40 min #3	N/A	13 sec	

Icon:

S

M

L

...